PRODUCT CONCEPT FOUNDATION | VERSION 1.0

CollectIQ

AI Receivables Operations Copilot

A detailed problem, solution, product strategy, architecture, and implementation brief for a privacy-conscious system that helps small businesses manage receivables without damaging customer relationships.

PRIMARY POSITIONING	Receivables operations and payment-relationship assistant
PRODUCT CATEGORY	Cross-platform receivables workflow overlay - not an invoicing system or debt collection agency
INITIAL CUSTOMER HYPOTHESIS	Small B2B agencies, consultancies, and service businesses with recurring invoices and mixed payment methods
STATUS	Pre-validation concept. Technical feasibility, customer demand, pricing, and provider approvals remain to be tested.
PREPARED	03 November 2026

THE PRODUCT IN ONE SENTENCE

Core promise: CollectIQ helps small businesses understand what is happening with outstanding invoices, reconcile payments across channels, interpret customer replies, and take the next appropriate action - while the business owner retains control.

This brief is intended to act as a source of truth for creating the downstream product documents. Treat all unvalidated assumptions as hypotheses, not established market facts.

START HERE

How to use this brief

This is the foundational concept document, not the final PRD, technical specification, or legal compliance assessment.

The purpose of this brief is to align the team on the problem, product boundary, user promise, trust model, and implementation sequence before producing detailed design and engineering documents. It captures the sixteen challenge questions raised during product evaluation and converts them into product decisions and validation tasks.

Recommended reading order

- Read Sections 1-4 to understand the product thesis, problem, target customer, and proposed solution.
- Read Sections 5-8 for the core workflow, architecture, AI boundaries, and privacy/security principles.
- Read Section 9 for all sixteen challenge questions and their proposed answers.
- Read Sections 10-13 for the build plan, metrics, risks, open decisions, and the seven downstream documents.
- Use the references in Section 14 to validate provider-level integration details before implementation.

Contents

1. Executive summary and product thesis - The short product definition and what makes the concept worth testing.

2. Problem statement and customer - Who experiences the problem, when it happens, and why current workflows are fragmented.

3. Proposed solution and product boundaries - What CollectIQ does, what it does not do, and the intended product principles.

4. Core workflow and user journeys - The end-to-end loop from invoice intake to verified payment and customer response.

5. Differentiation and competitive position - The potential moat versus scheduled reminders and existing finance tools.

6. AI and decision architecture - Where AI is genuinely useful, what stays deterministic, and how cold start is handled.

7. System architecture and data model - The services, states, integrations, and core data entities required.

8. Privacy, security, ethics, and trust - Data handling, relationship safeguards, and boundaries on autonomous actions.

9. Answers to the sixteen product challenges - Direct answers and design implications for every question raised.

10. MVP scope and solution implementation plan - Build phases, hackathon demonstration, and team workstreams.

11. Success metrics and validation plan - What to measure before claiming value or predictive capability.

12. Risks, dependencies, and open decisions - Known unknowns that need proof before product expansion.

13. Seven downstream documents - A proposed document set to derive from this concept foundation.

14. References and source notes - Current official provider and policy references used for integration cautions.

Working-name note: CollectIQ is provisional. Check trademark and domain availability before public use; avoid 'Invoice Recovery' as the primary positioning because it suggests debt collection.

01 / PRODUCT THESIS

Executive summary and product thesis

THE CORE PRODUCT THESIS

CollectIQ is an AI-assisted receivables operations overlay for small B2B service businesses. It connects the state of invoices, payment evidence, and customer email conversations so the owner can resolve exceptions and follow up appropriately without replacing the invoicing system or turning customer communication into aggressive debt collection.

The problem is not a lack of reminder buttons

Most businesses can already schedule a reminder in an accounting or invoicing tool. The harder operational problem is that the truth about an invoice is scattered. The invoice may have been created in accounting software or a document template; the payment may arrive through PayPal, UPI, NEFT, bank transfer, cash, or another provider; and the explanation for a delay may exist in an email thread. A reminder engine that sees only an invoice due date can miss this context.

The product opportunity is to coordinate the full receivables workflow: identify what needs attention, assemble relevant evidence, understand the customer's latest response, verify whether payment has arrived through any supported route, and recommend or execute only an authorized next action.

Product definition

CollectIQ is a cross-platform receivables workflow layer. It is not intended to become another invoice editor, accounting ledger, payment processor, credit bureau, or collection agency. It should coexist with the tools a business already uses and import or synchronize only the information required to manage receivables.

One-line promise

Understand every outstanding invoice, reconcile payments wherever they arrive, and respond to customers intelligently - without sacrificing the relationship or surrendering control.

The product's three pillars

PILLAR	WHAT IT MEANS
Payment truth	Reconcile supported PayPal events, bank-statement imports, manual confirmations, and other supported records. Distinguish verified paid, partially paid, open, disputed, and uncertain states.
Conversation understanding	Interpret relevant customer replies: payment timing, payment sent, dispute, missing documents, request for clarification, partial-payment proposal, or opt-out.
Safe orchestration	Use a deterministic policy engine to schedule, snooze, suppress, or escalate actions. Automate only narrow, reversible actions that the user has explicitly authorized.

Why now / why build it?

The case to test is not that AI can write a better reminder. The case is that AI can reduce the manual effort required to interpret unstructured replies and coordinate decisions across disconnected systems. Whether this is valuable enough to justify a separate product is an open market question and must be demonstrated in pilots.

02 / PROBLEM

Problem statement and target customer

Formal problem statement

Small B2B service businesses manage receivables across disconnected systems: invoices in accounting tools, spreadsheets, or documents; communication in email; and payments through PayPal, bank transfer, NEFT, UPI, cash, or other channels. Scheduled reminders may not see the latest payment evidence, customer replies, disputes, partial payments, or agreed dates. Owners therefore search threads, reconcile records, and decide manually whether to follow up - consuming time and risking unnecessary or poorly timed messages.

The operational gap

SOURCE	WHAT IT KNOWS / WHAT IT MISSES
Invoice system	Knows invoice number, due date, balance, and customer; may not know the latest conversation or payment outside that platform.
Email inbox	Contains payment commitments, disputes, questions, and context; does not reliably update invoice status.
Payment channels	Contain transaction evidence; records may not link cleanly to invoice numbers or payer identities.
Owner / bookkeeper	Acts as the human integration layer: searches, reconciles, interprets, and updates each system manually.

Initial customer segment hypothesis

Start with agencies, consultancies, studios, and B2B service providers with recurring invoices and repeated exceptions. Roughly 20-200 invoices per month is a hypothesis, not market research. Qualify by multiple payment routes, email-based clarifications, and meaningful reconciliation work. Very low-volume freelancers may need only Gmail and a spreadsheet; large enterprises may require ERP, vendor-onboarding, tax, and compliance capabilities beyond an early MVP.

Likely users and stakeholders

- Owner / founder: visibility, less administration, and protected client relationships.
- Finance admin / bookkeeper: accurate queues, reliable reconciliation, and audit history.
- Account / project lead: delivery context and authority to pause a disputed follow-up.
- Customer / AP contact: accurate, respectful communication and a clear way to explain or correct an issue.

Jobs to be done

- Show me which invoices genuinely need attention, not merely which due dates have passed.
- Tell me whether a payment may already have arrived through another channel.
- Understand the customer's reply and update the next step without making me reread every thread.
- Resolve missing information or disputes before another reminder, while keeping communication in my own mailbox and the relationship context intact.
- Automate narrow, reversible actions while keeping sensitive decisions under human control.

Customer qualification should be based on invoice volume, number of payment methods, exception frequency, and time spent reconciling - not on the broad label 'freelancer'.

03 / PROPOSED SOLUTION

What CollectIQ should be

Solution overview

CollectIQ sits above a business's existing invoicing and payment systems. It imports invoices and payment evidence, links records to customer email conversations, and builds a unified receivables work queue. AI interprets unstructured content and proposes structured events or next actions. A deterministic policy layer checks payment state, permissions, contact limits, dates, and exceptions before any action is scheduled or sent.

The product modules

MODULE	RESPONSIBILITY
1. Invoice intake	Import from CSV first; accept PDF/document upload for extraction with confirmation; add accounting-platform integrations only after API and permission validation. Do not require invoice creation inside CollectIQ.
2. Unified receivables ledger	Normalize invoice identifiers, customer references, amounts, currency, due dates, paid amounts, remaining balance, source, and evidence freshness.
3. Payment reconciliation	Ingest verified PayPal events, supported bank-statement exports, other supported gateway records, and user-entered payments. Match conservatively and preserve uncertain states.
4. Connected mailbox	Use OAuth to send from the user's own Gmail or Outlook mailbox. Add only the mailbox-reading permission required to detect relevant replies, with clear user consent and provider review.
5. Conversational intelligence	Classify customer replies, extract dates and payment references, identify disputes or questions, and generate structured interpretations with confidence and evidence.
6. Policy and workflow engine	Apply user-configured rules for snoozing, reminder intervals, suppressions, human approvals, and escalation. LLM output cannot bypass this engine.
7. Exception-first dashboard	Prioritize unresolved payment claims, disputes, ambiguous matches, missing information, overdue invoices, and scheduled commitments. Do not force users to inspect every invoice equally.
8. Audit and privacy layer	Record the evidence and policy behind each action, restrict access by tenant and role, minimize data sent to AI, and support export and deletion workflows.

What the product will not do

- It will not force users to create invoices inside CollectIQ or replace accounting software as the system of record.
- It will not force every customer to pay through PayPal. PayPal is one supported payment source and an optional payment route where appropriate.
- It will not claim that an invoice is unpaid solely because a PayPal event was not received.
- It will not autonomously make legal threats, add penalties, change contract terms, negotiate binding settlements, or escalate a dispute.
- It will not promise a 100% accurate interpretation of natural language or a 100% safe communication outcome.
- It will not claim enterprise certification, zero data retention, or regulatory compliance without verified evidence.

04 / CORE WORKFLOW

End-to-end operating model

The core loop

STAGE	OPERATION
Observe	Collect invoice data, current payment evidence, mailbox events, customer preferences, and existing commitments.
Normalize	Link records to a business, customer, invoice, currency, and source. Preserve provenance and freshness.
Understand	Use AI to extract intent, dates, references, disputes, questions, and payment claims from relevant text.
Verify	Use deterministic rules and trusted evidence to update balances and assess whether the invoice is paid, partially paid, open, disputed, or uncertain.
Decide	Run the proposed action through policy: consent, permissions, confidence, due date, frequency limits, dispute state, and payment status.
Act	Schedule, snooze, suppress, draft, send through the user's mailbox, or route to a human.
Learn	Record whether the action led to a verified payment, a clarified dispute, a changed commitment, or no response. Do not train models on client data without a clear lawful basis and appropriate controls.

Example user journey A: payroll timing reply

An owner has enabled automatic snoozing for unambiguous customer payment-timing statements. A customer replies, 'We run payroll on the 15th.' CollectIQ links the reply to the correct thread and invoice, classifies it as a payment-timing signal rather than a confirmed payment, extracts the likely date, checks timezone and month context, and validates that the date is unambiguous. If the context is clear and the policy permits, it moves the next routine reminder to the 16th. If the month is unclear, the reply conflicts with another commitment, or the invoice is disputed, it pauses and asks the owner to review. The activity is recorded in the audit trail.

Example user journey B: UPI or bank transfer

A customer says, 'I transferred the amount this morning.' CollectIQ does not simply snooze and does not mark the invoice paid based on the sentence alone. It looks for a matching imported transaction or supported bank record. A unique, high-confidence match can be proposed or auto-applied only under a configured policy. Ambiguous matches, partial amounts, duplicate amounts, or incomplete statement coverage are marked as uncertain and suppress routine reminders until checked.

Example user journey C: deliverable dispute

A customer replies, 'Section 3 was not completed. Please revise it before payment.' The system classifies this as a service or deliverable dispute, pauses routine payment reminders, links the message to the invoice and relevant project reference where available, summarizes the issue, and routes it to the owner or project lead. It must not continue sending reminders just because the due date has passed.

Invoice status and follow-up status must be separate

A single status field will create confusion. The invoice's financial state should be separate from the communication/workflow state. For example, an invoice can be 'partially paid' while its follow-up state is 'paused pending customer clarification'. This separation prevents an email event from incorrectly changing a financial balance.

STATE DIMENSION	EXAMPLES
Financial state	Open / outstanding; Partially paid; Paid - verified; Payment uncertain; Disputed / under review; Cancelled or written off if supported by source system.
Workflow state	Not contacted; Draft ready; Scheduled; Snoozed until date; Waiting for customer; Needs owner review; Suppressed; Resolved.
Evidence state	Verified provider event; imported transaction; user confirmation; inferred / candidate match; missing or stale evidence; conflicting evidence.

Pre-send safety gate

- Refresh the latest available payment and mailbox evidence; check source freshness and integration health.
- Cancel or suppress the action if the invoice is verified paid, cancelled, disputed, or under review.
- Pause if payment status is uncertain, a payment claim is awaiting reconciliation, or statement coverage is incomplete.
- Check customer commitments, snooze dates, opt-outs, wrong-recipient signals, contact limits, and business-hour settings.
- Validate the draft facts, amount, currency, recipient, invoice reference, and sender mailbox.
- Confirm that the action is permitted by the user's automation mode; enforce an idempotency key before queueing or sending.

NON-NEGOTIABLE PAYMENT RULE

Critical invariant: 'No event received' is not equivalent to 'not paid'. If payment status cannot be established with adequate evidence, label it uncertain and suppress automated contact until the uncertainty is resolved or a specifically authorized safe rule applies.

05 / DIFFERENTIATION

Competitive position and potential moat

Why the automated reminder checkbox is not enough

A scheduled reminder is a calendar rule: when the due date or aging threshold is reached, send a predefined message. That is useful and should not be dismissed. CollectIQ cannot justify a separate product merely by sending the same reminder with an LLM-written sentence. Existing platforms such as QuickBooks, Zoho Books, and FreshBooks may already provide some combination of reminder scheduling, invoice tracking, payment status, and customer communication. The exact feature coverage, plan restrictions, and regional availability must be checked in a current competitor matrix before product claims are made.

The proposed differentiation is the workflow between systems: combine invoice state, payment evidence from multiple rails, and customer correspondence; then handle exceptions such as a disputed deliverable, an unclear payment reference, a promised payroll date, or a partial payment. The system should reduce the number of invoices that require manual investigation and reduce incorrect follow-ups.

Potential differentiation layers

LAYER	WHY IT MAY MATTER
Cross-system view	Receivables overlay that links invoice records to payment sources and mailbox conversations without requiring a system replacement.
Exception handling	Detects and routes disputes, payment claims, partial payments, missing PO or tax details, wrong invoice references, and uncertain matches.
Conversation-to-workflow	Turns natural-language replies into structured commitments, snoozes, holds, reconciliation tasks, and owner actions.
Payment safety	Uses evidence-aware matching and a separate uncertain state to avoid treating missing webhooks as proof of non-payment.
Relationship controls	Honors client sensitivity, contact frequency limits, preferences, dispute states, and human approval boundaries.
Outcome evidence	Measures manual handling time, false reminders avoided, unresolved exceptions, and payment-resolution time. This is an operating advantage only if pilots demonstrate improvement.

What is and is not a moat

Integrations alone are not a durable moat. An LLM wrapper is not a moat. A dashboard of overdue invoices is not a moat. The moat hypothesis is a high-quality, trusted exception-resolution workflow that becomes more useful as it connects more of the business's receivables evidence and reliably turns customer replies into correct next steps. That claim is only credible after measured customer outcomes, strong integrations, reliable matching, and user trust have been established.

Competitive alternatives to test

- Existing accounting/invoicing software: QuickBooks, Zoho Books, FreshBooks, and regionally popular alternatives.
- The status quo: Gmail or Outlook templates, spreadsheet trackers, calendar reminders, and manual bank-statement review.
- Bookkeepers or outsourced finance administrators who provide human reconciliation and follow-up.
- Standalone accounts-receivable tools and collections workflows serving larger businesses.

06 / AI AND DECISIONS

Where AI is substantive - and where it is not

A clear division of responsibility

COMPONENT	APPROPRIATE RESPONSIBILITY
Deterministic application logic	Calculate balances; compare dates; enforce schedules; prevent duplicate actions; apply contact-frequency limits; verify webhook signatures; maintain permissions; determine whether an invoice is eligible for an action.
LLM / language model	Parse invoice and email text; classify customer intent; identify disputes or questions; extract candidate dates, payment references, and commitments; summarize threads; draft tone-appropriate responses; explain recommended next steps.
Statistical / ML models (later)	Estimate operational patterns only after adequate, representative, permissioned data exists: likely late-payment timing, response likelihood, or workload prioritization. Models must be validated and should not make creditworthiness or default claims without a justified use case and governance.
Human judgment	Resolve disputed deliverables, ambiguous commitments, unusual client relationships, proposed payment arrangements, legal or contractual questions, and exceptions that exceed configured autonomy.

Why an LLM is needed at all

A database query can identify an overdue invoice. It cannot reliably interpret a message such as 'We run payroll on the 15th', 'The PO number is incorrect', 'We paid this through the other account', or 'Please hold payment until Section 3 is fixed'. These messages are unstructured, contextual, and may refer to prior conversations. Language understanding is a real use for an LLM: extract the intent and relevant facts into a structured event that the application can validate.

The LLM does not get authority to update the ledger or send an email merely because it produced a JSON object. The output is a proposal. Deterministic validators check the invoice association, date interpretation, allowed action, evidence, user policy, and required approval.

Cold start: what happens with zero historical data?

On day one, CollectIQ does not know whether a particular customer will pay late or default. It must not present a predictive default probability or a precise 'cash-flow risk score' as if it were statistically supported. It can provide descriptive facts and transparent rules: invoice is 12 days overdue; no payment is matched in the imported statement period; customer said payment is planned for the 15th; invoice is disputed; or transaction matching is uncertain.

A first version may use explainable priority indicators rather than prediction. Examples include days overdue, remaining balance, customer-reported payment timing, unresolved dispute, missing purchase-order reference, previous manually recorded commitments if supplied by the user, and evidence freshness. Each indicator must display its reason. This is an operational queue, not a credit score.

How predictive models could be introduced later

- Collect sufficient historical outcomes only with appropriate notices, permissions, data governance, and contractual controls.
- Define the prediction target precisely: for example, 'probability of payment within 14 days' is different from 'default risk'. Avoid vague labels.
- Use temporal validation and out-of-time testing; avoid data leakage from future payment outcomes.

- Check calibration across customer segments, invoice sizes, currencies, industries, and regions; monitor drift.
- Provide explanations and conservative fallbacks. Never let a prediction alone trigger threats, penalties, or high-pressure contact.

AI output contract

For every interpreted reply, the model should return a constrained structure such as: intent category; invoice/thread candidate; extracted date or payment reference; whether the statement is a confirmed fact, proposal, or vague indication; confidence; evidence excerpt; recommended action; and reasons for escalation. The application validates every field. Unknown or ambiguous values must remain unknown rather than being silently guessed. This is the substantive AI role: turning unstructured client communication into a candidate workflow event. SQL, transaction matching, date arithmetic, and permission checks remain deterministic.

Illustrative structured interpretation

FIELD	ILLUSTRATIVE VALUE
Incoming text	We run payroll on the 15th.
Intent	Payment-timing signal; not proof of payment.
Date extraction	15th of the relevant month; month must be resolved from thread context or remain unknown.
Commitment strength	Indicative timing, not a guaranteed payment commitment.
Candidate action	Snooze routine reminder until the 16th, only if date context is unambiguous.
Policy checks	Correct invoice/thread; no active dispute; no payment uncertainty; user enabled automatic snoozing; contact limits satisfied.
Fallback	If month, invoice, or meaning is ambiguous, do not guess. Pause and request owner review.

07 / SYSTEM DESIGN

Reference architecture and core data model

Logical architecture

The following is a reference architecture for planning. It is not a claim that these services have been built or that every integration is already available to the product.

LAYER	RESPONSIBILITY
User interface	Receivables queue, invoice detail, payment evidence, connected mailbox status, approval controls, policy settings, audit history.
Backend API and identity	Tenant membership, user authentication, role checks, OAuth connection lifecycle, API authorization, rate limits.
Ingestion adapters	CSV import, PDF/document extraction, PayPal API/webhook adapter, Gmail adapter, Microsoft Graph adapter, manual payment form.
Canonical receivables service	Normalize invoice, customer, currency, balance, due date, source, and evidence provenance.
Reconciliation engine	Candidate matching, partial payments, duplicate detection, confidence bands, uncertain-state handling, human confirmation.
Conversation intelligence service	Thread association, text minimization, intent extraction, date interpretation, dispute classification, draft generation.
Policy and scheduler	Allowed actions, snooze rules, communication limits, consent, confidence thresholds, approval requirements, idempotent jobs.
Persistence and audit	Encrypted relational database, protected object storage, token vault/secret manager, action log, retention and deletion controls.
AI provider gateway	Provider abstraction, prompt minimization, redaction, timeout/retry policy, model routing, output validation, vendor policy configuration.

Core data entities

ENTITY	KEY FIELDS / PURPOSE
Business / Tenant	Organization boundary, settings, region, timezone, data-retention policy, connected integrations.
User / Role	Owner, finance admin, bookkeeper, project lead, reviewer; permissions and approval limits.
Client / Contact	Customer identity, approved email addresses, relationship category, preferred tone, contact rules, opt-out/sensitivity flags.
Invoice	External ID, source system, customer, issue/due date, currency, gross amount, paid amount, outstanding balance, invoice reference, status.
Payment Transaction	Provider or import source, reference, payer descriptor, amount, currency, timestamp, settlement/status, raw-source pointer.
Reconciliation Match	Candidate invoice, transaction, match factors, score/confidence, evidence, confirmed/rejected/needs-review state.
Mailbox Connection	Provider, scopes, encrypted token reference, mailbox identity, subscription/watch state, expiry, connection status.
Email Thread / Message	Provider IDs, sender/recipient, timestamps, thread link, minimal content pointer, classification, processing status.
Customer Commitment	Candidate or confirmed payment date, amount if present, confidence, source message, snooze state, created/validated time.
Dispute / Exception	Type, description, related message, affected invoice, assigned owner, current resolution state.
Follow-up Policy / Scheduled Action	Allowed action, condition, run time, timezone, idempotency key, approval requirement, cancellation state.
AI Interpretation / Audit Event	Model/provider version, fields processed, structured result, validation outcome, policy decision, user override, action evidence.

State machine and invariants

- Financial status and follow-up status are independent. Email actions must never directly mark an invoice paid.
- An event is idempotent: webhook retries or repeated imports must not create duplicate payments or duplicate reminders.
- A payment match must retain its evidence and confidence. A user override must be auditable and attributable.
- Every scheduled action is revalidated immediately before execution against the latest payment, dispute, consent, and policy state.
- When evidence is stale, missing, contradictory, or ambiguous, automated contact is paused or held for review.
- Every integration must have a disconnect/revocation path and a safe response when tokens or subscriptions expire.

PayPal integration boundary

PayPal webhooks can notify an application about subscribed event types, and the webhook message must be verified before processing. PayPal's documentation also explains that events are associated with the specific REST application used for the integration. Therefore, the team must prove which transactions are visible for an authorized merchant connection and must not assume that subscribing a webhook automatically yields a complete history of every payment on every PayPal account. Use sandbox tests, verify signatures, and reconcile against the supported API data available to the connected merchant.

Email provider boundary

Sending mail and reading incoming replies are different permissions. Gmail's send method supports an OAuth scope specifically for sending; Google classifies gmail.send as a sensitive scope. Reading message content may require additional sensitive or restricted scopes, depending on the implementation. Gmail push notifications use Cloud Pub/Sub, require watch renewal, and can occasionally be delayed or dropped, so a sync fallback is needed. Microsoft Graph offers a sendMail API with Mail.Send; message change notifications require read access to the message resource. Provider consent, application verification, notification lifecycle, throttling, and subscription renewal are engineering work items, not assumptions.

Mailbox integration proof checklist

- Prove send-only operation first; do not request inbox reading access just to send a reminder.
- Test reply association using provider message and thread identifiers, not subject text alone.
- Document the exact OAuth scopes and review requirements for the chosen app distribution model.
- Handle watch/subscription renewal, expired tokens, rate limits, retries, notification loss, and periodic synchronization.
- Expose connection status clearly: connected, needs reauthorization, degraded monitoring, or disconnected.
- On disconnect, stop future sending and reply monitoring, revoke tokens where supported, and preserve the user's ability to export records.

08 / TRUST AND SAFETY

Privacy, security, ethics, and relationship safeguards

Data classification

DATA CLASS	EXAMPLES
Identity and contact	Client names, email addresses, phone numbers, company identities.
Commercially sensitive	Hourly rates, invoice amounts, discounts, project scope, service descriptions, PO numbers, contracts and NDA-related content.
Financial evidence	Bank-statement rows, payment references, payer descriptors, transaction amounts, payment dates, PayPal identifiers.
Communication content	Customer replies, dispute explanations, attachments, commitments, and internal notes.
Credentials and tokens	OAuth refresh tokens, API credentials, webhook secrets, encryption keys. Highest protection; never expose to the LLM or frontend.

Privacy-first AI processing

A hosted LLM API is not automatically a public bulletin board, but sending data to a hosted model provider is still external processing by another service provider. Privacy depends on the exact provider, product tier, contract, retention configuration, security controls, region, subprocessors, and data-use terms. Do not tell customers that data is 'strictly private', 'never retained', or 'never used for training' unless the selected provider's actual terms and configuration substantiate that statement.

Recommended design: keep raw invoices and full customer records in the private application data layer; create a purpose-specific AI payload; remove direct identifiers and irrelevant pricing or project details where possible; process only the smallest relevant text span; validate output; and store the interpretation with source references and a retention policy. Redaction reduces exposure but does not guarantee anonymization, because free text and combinations of details can re-identify a client.

Minimum security controls

CONTROL	REQUIREMENT
Encryption	TLS in transit; strong encryption at rest; managed keys and rotation procedures.
Tenant isolation	Every database query and object access must be scoped to the business tenant; test cross-tenant access explicitly.
Access control	Least privilege, role-based permissions, owner approval policies, MFA support, and secure session handling.
OAuth token safety	Encrypt refresh tokens; restrict scopes; rotate or revoke on disconnect; never return secrets to frontend code.
Secret management	Store API keys, webhook verification material, and signing secrets in server-side secret management.
Logging	Do not place full invoice bodies, bank data, credentials, or customer emails into general application logs or analytics.
Retention and deletion	Define retention periods for files, extracted text, prompts, embeddings, caches, logs, backups, and derived records; implement deletion workflows.
AI input safety	Treat customer email and invoice text as untrusted data; resist prompt injection; never interpret embedded instructions as permission to access tools or override policies.
Audit trail	Log who or what accessed data, what the system proposed, the policy result, what action occurred, and who approved it.
Incident readiness	Document detection, containment, notification, recovery, and customer support procedures before production use.

Relationship-safe automation

- No threatening, shaming, deceptive, or manipulative message styles. Do not optimize the system to maximize contact pressure.
- Respect contact-frequency limits, business hours, customer preferences, agreed snooze dates, and wrong-recipient or opt-out signals.
- Pause routine reminders when a customer disputes the work, challenges the amount, asks for clarification, claims payment, or reports a sensitive circumstance.
- Do not invent legal action, late fees, contractual rights, or payment deadlines. The product must not provide legal conclusions.
- Offer owner-configurable client relationship profiles and tone controls, but do not let tone preference override factual accuracy or customer contact restrictions.
- Every automated behavior must be narrow, reversible, permissioned, and explainable. The owner must be able to pause or disconnect automation immediately.

Privacy and compliance posture

The correct pre-launch statement is 'privacy-first architecture designed to minimize sensitive data exposure', not 'enterprise-grade compliant'. A product operating in India should obtain legal advice on the Digital Personal Data Protection Act, 2023 and the DPDP Rules, 2025, including the specific commencement, applicability, notice, consent, processor, security, retention, and rights requirements relevant to its actual processing. The Government of India announced notification of the DPDP Rules on 14 November 2025; the precise obligations and implementation timeline should be checked with counsel rather than inferred from a marketing checklist [8]. Contractual confidentiality, international data transfers, email-provider terms, and other applicable laws may also matter.

The product cannot be 100% safe. Its defensible promise is controlled risk: minimize data, limit permissions, pause under uncertainty, keep humans involved in consequential cases, record actions, and test failure scenarios.

09 / CHALLENGE REVIEW

Answers to the sixteen product challenges

These are product decisions and proposed answers, not claims that the implementation already exists.

1. Why not just turn on automated reminders in QuickBooks, Zoho, or FreshBooks?

Because a reminder checkbox addresses only scheduled communication. It does not, by itself, solve the cross-system problem of linking invoice status, payment evidence from other channels, customer replies, partial payments, and disputes. However, the existence of those broader features in competitors must be checked. CollectIQ earns a place only if it demonstrably reduces manual exception handling beyond what the customer's current tools already do.

- Position as an overlay, not a new invoice system.
- Build an exception-first queue rather than another reminder scheduler.
- Prove value using time saved, reconciliation accuracy, and unnecessary reminders avoided.
- Benchmark actual competitor features, pricing tiers, integrations, and regional availability before making comparison claims.

PRODUCT DECISION

The moat hypothesis is trusted cross-system exception resolution, not AI-generated reminder text. Integrations are not enough unless they produce measurable outcomes.

2. Is an LLM really needed, or is this rule-based logic dressed up as AI?

Due-date checks, arithmetic, balance calculations, status transitions, permission checks, matching rules, and reminder eligibility are deterministic application logic. An LLM is justified for interpreting unstructured messages and documents: a customer's statement about payroll timing, a payment claim, a PO-number problem, a deliverable dispute, or a request to hold payment. The LLM converts language into a structured candidate event; code validates it and controls the action.

- Use SQL and deterministic code for dates, balances, reconciliation rules, and enforcement.
- Use the LLM for reply classification, date/reference extraction, dispute and intent detection, thread summaries, and draft suggestions.
- Require schema validation, confidence handling, source evidence, and policy checks.
- Escalate ambiguous or high-impact interpretations instead of guessing.

PRODUCT DECISION

AI value is language-to-workflow interpretation. The system must not label ordinary database logic as machine learning.

3. How can it evaluate cash-flow risk with zero historical data?

It cannot reliably predict client default or payment delay from no history. The day-one product should not show a fabricated probability or imply a credit-risk model exists. It can display transparent operational indicators: days overdue, amount outstanding, missing payment evidence, a customer-stated timing signal, an open dispute, or an ambiguous transaction match. These are facts and rules, not a predictive risk score.

- Use an exception-priority indicator with visible reasons, not a default probability.
- Allow an owner to import historical data, but do not silently treat incomplete data as representative.
- Introduce a predictive model only after sufficient permissioned, representative outcomes exist.
- Validate with out-of-time data, calibration, subgroup analysis, drift monitoring, and a clear prediction target.

PRODUCT DECISION

Cold-start rule: no historical data means no defensible prediction. Be transparent about what is known and unknown.

4. What is the customer acquisition strategy?

Do not target every freelancer. Begin with a narrow segment of agencies, consultancies, studios, or B2B service providers whose recurring receivables work creates repeated reconciliation and communication overhead. Start with founder-led discovery and pilots, then test bookkeeper/accountant referrals, integration directories, professional communities, and referral loops. Paid acquisition should wait until retention and willingness to pay are evidenced.

- Interview 15-20 qualified businesses and observe their current workflow; numbers are a proposed discovery target, not completed research.
- Recruit a small pilot cohort and measure before/after workflow time and error rates.
- Test accountant/bookkeeper partnerships because one trusted intermediary may refer multiple clients.
- Test pricing through real willingness-to-pay conversations and paid pilots; indicative pricing such as INR 999-2,499/month is only a hypothesis, not a validated price.

PRODUCT DECISION

The business case requires measurable recurring value and acceptable acquisition economics. Low-volume users may be better served by Gmail and a spreadsheet.

5. Will this make me look desperate or damage an important client relationship?

No product can guarantee that every interpretation or message will be received well. CollectIQ should be a relationship-aware assistant, not an autonomous debt collector. It should understand the customer's context, pause when a dispute or sensitive situation appears, use user-configured tone and contact limits, and require review for ambiguous or consequential cases.

- Create client sensitivity settings such as standard, long-term/high-value, and sensitive; treat these as user-entered guidance, not AI-inferred personality profiles.
- Default to human approval while users build trust; later allow narrowly bounded automation for routine cases.
- Pause on disputes, payment claims, uncertainty, opt-outs, and requests to hold contact.
- Prohibit threats, fake urgency, invented penalties, and legal escalation language.

PRODUCT DECISION

Promise risk reduction and control, not 100% safety. The user can always pause automation and correct the system.

6. Do I have to re-enter all client and invoice details?

No. Re-entry would make adoption harder and would turn CollectIQ into a competing system of record. It should import from the user's existing workflow. For an MVP, prioritize CSV/spreadsheet import and PDF upload or email-forwarding with extraction and user confirmation. Add selected accounting integrations only after their APIs, permissions, and plan limitations are validated.

- Never require invoice creation inside CollectIQ.
- Preserve source IDs and original invoice references so records can be traced back.
- Show low-confidence extracted fields for confirmation.
- Avoid broad integrations until there is evidence which source systems target customers actually use.

PRODUCT DECISION

Overlay first: import and coordinate, do not duplicate the user's invoicing workflow.

7. What if the customer pays by bank transfer, cash, or UPI instead of PayPal?

PayPal must be one payment source, not the definition of payment truth. The MVP should combine verified PayPal events with bank-statement CSV imports and manual cash/UPI/bank confirmations. Matching should use reference, amount, currency, date, payer descriptor, and invoice context. If the match is ambiguous, the status remains uncertain and routine reminders pause.

- Support partial payments and duplicate transaction detection.
- Record evidence source, import period, match confidence, and who confirmed it.
- Do not mark paid from an unverified customer statement alone.
- Do not treat absence of a PayPal webhook as proof that the invoice is unpaid.

PRODUCT DECISION

A safe reconciliation engine is more important than broad payment-rail claims. Start with CSV and manual evidence; validate live bank feeds later.

8. How much time does it save if I still approve every draft? Why not Gmail templates?

Gmail templates help write or send a message. CollectIQ must save work before and after the email: find what needs attention, check payment evidence, identify exceptions, understand customer replies, track commitments, update invoice context, and suppress unnecessary contact. Approval remains necessary for sensitive messages, but low-risk actions such as an explicitly authorized snooze can run automatically. A low-volume freelancer may not need this product.

- Build an exception-first queue so the owner sees only invoices needing judgment.
- Offer approval modes: manual review, batch review of low-risk drafts, and bounded automation for authorized low-risk actions.
- Automatically snooze only when the extracted date is unambiguous and the policy explicitly permits it.
- Measure minutes per invoice, human interventions per 100 invoices, false reminders avoided, and time to resolve exceptions.

PRODUCT DECISION

The product saves time by coordinating the workflow, not by composing faster emails. Any time-saved number shown before pilots must be labeled illustrative.

9. Who is this email coming from?

It should come from the user's connected business or personal mailbox, not a generic CollectIQ sender, for the primary customer-facing reminder workflow. Use OAuth and the provider's sending API. This preserves the user's sender identity and Sent Items history, subject to provider rules and permissions. Inbox placement is not guaranteed: spam filters, sending limits, domain reputation, and recipient policies still apply.

- Start with one provider integration and a dedicated test mailbox; add the second provider after the flow is stable.
- Use the narrowest send permission possible and obtain separate read permission only for reply ingestion.
- Store OAuth tokens securely, support revoke/disconnect, and keep message/thread IDs for correlation.
- Test delivery failures, provider throttling, message threading, and domain authentication behavior.

PRODUCT DECISION

Connected sender identity is a core requirement. It does not guarantee that messages land in the recipient's inbox.

10. Why is the payment link forcing PayPal? What about corporate NEFT/ACH/vendor onboarding?

It must not force PayPal. CollectIQ is not a payment processor and should not dictate how an invoice is settled. PayPal can provide verified payment evidence and an optional payment route where the business chooses it. The invoice's existing bank details, NEFT/ACH instructions, tax invoice, PO number, and vendor-onboarding workflow remain authoritative. The product should help track the chosen route rather than redirect every customer to PayPal.

- Do not rewrite payment terms or bank details without explicit user action.
- Preserve the original invoice and payment instructions.
- Support a structured payment-method field and references for bank transfer, UPI, cash, PayPal, and other methods.
- For corporate accounts payable, surface missing PO/vendor/tax information as an exception rather than sending a generic reminder.

PRODUCT DECISION

PayPal is an integration and optional rail, not a forced checkout requirement. This also keeps the system useful for mixed payment workflows.

11. What happens if a customer replies with a question or deliverable dispute?

The reply must be interpreted and associated with the invoice before another reminder is considered. A deliverable dispute, amount challenge, missing document, PO issue, or request for clarification should pause routine reminders and create an exception for the relevant owner. The AI can summarize the issue and suggest a response, but it must not decide whether work was contractually complete or whether payment is legally due.

- Use intent categories: payment timing, payment sent, dispute, question, missing documentation, partial payment, wrong recipient, opt-out, other/uncertain.
- Keep the original message and source thread reference available to authorized users.
- Pause when classification is uncertain or contradictory.
- Route project-delivery disputes to a project owner, not only the finance queue.

PRODUCT DECISION

Customer replies are state-changing inputs. A reminder engine that ignores them is not conversational intelligence.

12. Is this basically automated debt collection?

It could become that if the product is optimized to pressure customers into paying. The intended boundary is receivables operations and fair payment coordination. The system should help the business understand payment status, resolve blockers, and send accurate, respectful communications. It should not threaten, shame, impersonate legal authority, fabricate penalties, or increase pressure because a customer is late.

- Do not measure success by reminder volume or message aggressiveness.
- Support payment-date discussions and mutually agreed plans without binding either party automatically.
- Pause on disputes and sensitive circumstances; maintain user control.
- Avoid legal threats, collections escalation, credit reporting, or debt-sale workflows in the MVP.

PRODUCT DECISION

The product helps businesses get paid fairly while preserving relationships; it is not an automated debt collector.

13. Is financial and client-pricing data secure? Is it sent to a public LLM API?

The product should minimize what it sends to any external AI provider. A hosted LLM API is third-party processing, even when prompts are not publicly exposed. Actual privacy depends on the selected vendor, contract, retention settings, access controls, region, and data-use terms. CollectIQ cannot promise enterprise-grade compliance or zero retention until those properties are independently verified for the chosen stack.

- Keep raw invoice files and full commercial records in the private application store.
- Create task-specific redacted payloads and avoid sending unnecessary pricing, project scope, identity, or transaction data.
- Never send OAuth tokens, passwords, or secrets to the model.
- Review vendor training, retention, deletion, subprocessor, residency, and security terms; use contractual controls where appropriate.
- Build encryption, tenant isolation, least privilege, access logs, deletion, and incident response.

PRODUCT DECISION

Use the truthful launch description: privacy-first architecture designed to minimize sensitive data exposure; formal certification and legal compliance remain to be established.

14. Email deliverability and sender identity: use the user's Gmail/Outlook mailbox.

Agreed. Primary outbound mail should be sent through the user's connected mailbox using OAuth and the official provider API. Sending and reading replies should be treated as separate permissions. Mailbox access should be narrow, transparent, revocable, and limited to the features the user enabled.

- Gmail: use the Gmail API send method and request the least-privileged send scope appropriate to the use case; Google classifies gmail.send as sensitive [1][2].
- Outlook: use Microsoft Graph sendMail with Mail.Send; receiving message-change notifications requires read permission for the message resource [4][5].
- For Gmail push updates, use Pub/Sub and renew watches; notifications may be delayed or dropped, so implement sync fallback [3].
- Complete provider consent, verification, and application review where required; do not imply these approvals are automatic.

PRODUCT DECISION

Build mailbox connection, message threading, provider health, token lifecycle, and revocation as first-class product features.

15. Offline reconciliation: avoid contacting a customer who has already paid elsewhere.

Agreed. Build a payment-method-agnostic reconciliation workflow. PayPal webhooks can be one signal, but bank statements, UPI references, cash confirmations, and other gateway exports must also be considered. PayPal webhook authenticity must be verified before processing [6][7]. The system must preserve uncertain states when there is not enough evidence.

- MVP: bank-statement CSV import, manual payment record, partial payment, duplicate detection, and PayPal sandbox webhook verification.
- Match with invoice reference, exact/near amount, currency, date range, payer description, and known payment identifiers.

- Require confirmation for multiple candidates or low-confidence matches.
- Re-check invoice state immediately before any scheduled message is sent.

PRODUCT DECISION

The safety goal is not to always auto-match. It is to avoid a false assumption of non-payment and prevent an unnecessary reminder.

16. True conversational intelligence: parse 'We run payroll on the 15th' and snooze until the 16th.

This is a strong demonstration of the intended AI workflow. The LLM extracts a payment-timing signal and candidate date. The deterministic system resolves month, timezone, invoice association, confidence, existing commitments, and automation permission. If the message is unambiguous and the user has explicitly authorized automatic snoozing, CollectIQ can snooze the routine reminder until the 16th without manual approval. It should not describe the message as a guaranteed payment unless the customer explicitly committed to paying.

- Distinguish a payment window from a firm commitment.
- Resolve the month from context; if ambiguous, pause and ask the owner rather than guessing.
- Snooze is reversible and low-risk; only this explicitly authorized action should be automatic by default.
- If the reply says 'we paid yesterday', trigger reconciliation; if it contains a dispute, pause and escalate.
- Log the source message, extracted date, interpretation, policy decision, snooze result, and resume time.

PRODUCT DECISION

The demo should prove a complete loop: incoming reply -> extracted intent/date -> policy validation -> automatic snooze -> visible audit record.

10 / IMPLEMENTATION

MVP scope and solution plan

Recommended MVP outcome

Demonstrate that CollectIQ can import invoices, reconcile supported payment evidence, read a relevant customer reply, interpret the reply, and safely update the next action - without forcing PayPal or creating false payment certainty. The strongest demo is not a large dashboard. It is one trustworthy end-to-end workflow with evidence and policy controls visible.

MVP feature boundary

AREA	MVP DEFINITION
Invoice import	CSV import as the guaranteed path; PDF upload and extraction as a secondary path with field confirmation. No forced invoice creation.
Payment evidence	PayPal sandbox event path with signature verification; bank statement CSV; manual cash/UPI/bank payment entry; partial payments.
Mailbox	One provider first (Gmail or Outlook) in a controlled test account; connected sender identity; reply ingestion only with the required read permission and a working provider setup.
Conversational AI	Classify payment timing, payment sent, dispute, question, missing details, partial payment, and uncertain/other; extract candidate dates and references.
Policy engine	Configurable snoozing, dispute pause, uncertainty pause, reminder limits, explicit user opt-in for automatic snooze, idempotent jobs.
Dashboard	Exception-first work queue with financial state, evidence, conversation summary, recommended action, approval status, and audit history.
Privacy and security	Tenant isolation, encrypted secrets/tokens, minimized LLM payloads, protected logs, access controls, and deletion/disconnect flow.
Not in MVP	Multi-accounting integrations, live bank aggregation, predictive default model, autonomous negotiation, legal escalation, enterprise certifications, complex collections workflows.

Phased solution plan

PHASE	DELIVERABLE / EXIT CONDITION
Phase 0 - Validate the pain	Interview 15-20 target businesses; observe real anonymized workflows; quantify invoice volume, exceptions, admin time, payment methods, current tools, and willingness to pay. Confirm which customer segment feels pain frequently enough.
Phase 1 - Build the canonical receivables core	Define tenant, client, invoice, payment, evidence, commitment, dispute, action, and audit models. Build CSV import, status separation, idempotency, and the exception-first queue.
Phase 2 - Build payment reconciliation	Add manual payments and bank-statement CSV parsing. Add conservative matching, partial payments, duplicates, evidence traceability, and uncertain-state suppression. Add PayPal sandbox webhook path and signature verification.
Phase 3 - Build the email loop	Connect one test mailbox through OAuth. Send from the connected mailbox. Ingest relevant replies through the provider's supported mechanism. Add safe synchronization fallback, token lifecycle, and disconnect controls.
Phase 4 - Add conversational intelligence	Use an LLM to classify reply intent and extract dates/references. Validate against schemas and policy. Demonstrate payment timing snooze, payment claim reconciliation, and dispute pause.
Phase 5 - Harden and pilot	Run scenario tests, verify tenant boundaries, measure false reminders and manual effort, conduct user walkthroughs, document limitations, and only then test a small live pilot with clear consent and an override mechanism.
Phase 6 - Expand selectively	Add the second mailbox provider, one accounting integration selected by actual pilot demand, more bank feeds if feasible, and predictive analytics only when enough trusted data exists.

Three-person workstream suggestion

WORKSTREAM	RESPONSIBILITY
Member A - Backend and integrations	Canonical data model, invoice import, payment ingestion, reconciliation logic, PayPal sandbox/webhook verification, secure token handling.
Member B - AI and workflow intelligence	Reply intent taxonomy, LLM prompts/schema, date and reference extraction, confidence handling, policy interface, evaluation dataset.
Member C - Frontend and product experience	Exception-first dashboard, invoice detail, mailbox connection UI, review/approval controls, import flow, audit timeline, demo narrative.
Shared responsibility	Agree data contracts first; define test cases together; review security and unsafe states; integrate daily; prepare a single end-to-end demo.

Suggested hackathon demo script

1. Import a synthetic invoice from CSV. Show that no re-entry into a new invoice editor is required.
2. Show a PayPal sandbox payment event being received and signature-verified, plus an imported bank CSV with a separate transaction.
3. Show one invoice as payment-uncertain because two transactions could match; verify that the system suppresses a reminder.
4. Connect a dedicated test mailbox and show a reminder sent from the connected sender identity.
5. Reply as the customer: 'We run payroll on the 15th.' Show AI extraction, policy validation, automatic snooze to the 16th, and an audit entry.
6. Reply with a deliverable dispute. Show reminders paused and an owner review task created.
7. Finish by showing the privacy gateway payload: only necessary fields were passed to the LLM, with no secrets or full contract.

11 / VALIDATION

Success metrics and evidence required

North-star outcome

Reduce the manual effort required to resolve receivables exceptions while maintaining high payment-status accuracy and a low rate of unnecessary customer contact.

Product and operational metrics

METRIC	WHAT IT MEANS
False reminder rate	Share of reminders sent when a payment had already been made, a dispute was active, contact was paused, or evidence was uncertain. This should be treated as a critical safety metric.
Payment-status accuracy	Correctness of paid, partial, open, uncertain, and disputed states against manually verified ground truth.
Reconciliation quality	Precision of automatically applied matches; ambiguous candidate rate; percentage requiring human confirmation; duplicate-detection performance.
Conversation interpretation quality	Accuracy by intent class; date extraction accuracy; rate of unsafe auto-snoozes; escalation rate for ambiguous replies.
Human effort	Time spent per invoice or per 100 invoices; number of manual lookups; number of owner interventions; exceptions resolved per session.
Workflow outcome	Time from customer reply to correct action; time to resolve a dispute or missing information; commitments kept or missed.
Relationship quality	Customer complaints, wrong-recipient events, opt-outs, user-reported tone issues, and reversals of automated actions.
Commercial viability	Activation, weekly usage, retention, conversion to paid pilot, churn, acquisition cost, gross margin, and payback period.

Critical acceptance tests

- If a verified full payment is received, all future reminder jobs for that invoice are cancelled or re-evaluated idempotently.
- If a partial payment is received, the remaining balance is accurate and the reminder content cannot imply the original full amount remains due.
- If payment status is uncertain, no automatic reminder is sent by default.
- If a dispute is detected, routine reminder automation pauses and an owner review item is created.
- If a customer says 'We run payroll on the 15th', the system distinguishes timing information from a guaranteed promise and snoozes only under explicit policy and unambiguous date context.
- If the date, invoice, or customer thread is ambiguous, the system does not guess; it escalates or requests review.
- If a webhook is duplicated or retried, no duplicate payment or reminder is created.
- If an OAuth token is revoked or a mailbox subscription expires, the product shows a disconnected/degraded state and does not pretend reply monitoring is active.
- If an AI payload is generated, it contains only fields allowed for that task and never includes credentials or unnecessary invoice/contract information.
- If a user disconnects an integration or deletes data, the product performs the documented revocation/deletion workflow and records completion status.

Discovery and pilot plan

A proposed initial discovery cycle is 15-20 structured interviews with target businesses, followed by 3-5 hands-on pilots. This is a plan, not evidence that those interviews or pilots have already occurred. Ask participants to show their current process using sanitized examples. Measure actual time spent on invoice tracking, email search, payment matching, and follow-up decisions. Avoid relying only on stated interest or hypothetical willingness to pay.

During pilots, compare a defined baseline period with the CollectIQ-assisted period. Record invoice volume and complexity so that results are not confounded by different workloads. A business should be able to stop the pilot, export its data, and disconnect integrations easily.

12 / RISKS AND OPEN DECISIONS

Known risks, dependencies, and unresolved questions

RISK	LEVEL	MITIGATION
Competitor overlap	High	Run task-based competitor review; define a narrow workflow gap; do not claim missing features without evidence.
Low willingness to pay / churn	High	Select users with recurring workflow pain; validate with paid pilots; measure retention before scaling acquisition.
Mailbox permission and verification friction	High	Start with one provider and test-account setup; scope permissions narrowly; verify OAuth approval requirements early.
Incomplete payment visibility	High	Do not assume webhook coverage; use statement imports and manual evidence; maintain uncertain state; test account-level access.
False payment match	High	Conservative matching, duplicate detection, human confirmation for ambiguity, evidence provenance, and pre-send recheck.
LLM misclassification or hallucinated date	High	Constrained schema, evidence spans, confidence thresholds, deterministic checks, ambiguous-case escalation, evaluation test set.
Client relationship damage	High	Frequency limits, dispute pause, no threatening language, human review for sensitive cases, complaint monitoring.
Sensitive commercial data exposure	High	Data minimization, token vault, tenant isolation, vendor review, encryption, logging controls, retention and incident response.
AI prompt injection through customer text	Medium-High	Treat mail as untrusted input; isolate tools; never allow email content to alter permissions or policy; validate tool calls.
Operational dependency on provider APIs	Medium-High	Monitor quotas and errors; use sync fallbacks; retry idempotently; expose degraded integration state.
Compliance and contractual risk	High	Obtain legal advice; map data flows and roles; document vendor contracts, retention, notices, permissions, and customer rights.
Scope expansion	High	Keep MVP limited to CSV, manual payment, one mailbox, PayPal sandbox, and three core reply scenarios.

Decisions to make before the PRD

1. Is the initial launch market India-first, global-first, or an explicitly defined cross-border agency segment? This affects currencies, payment rails, tax documents, data handling, and customer acquisition.
2. Which first mailbox provider is most common among the chosen pilot customers: Gmail or Outlook? The answer determines the first OAuth implementation and app-review path.
3. Will CollectIQ read only designated invoice-related labels/folders, or will it process all inbox messages? The privacy and scope implications differ substantially.
4. What minimum level of payment evidence is required before an invoice can be marked paid or before an automated reminder can be sent? Define this per source and data freshness.
5. Which accounting/invoicing platform appears most often in customer interviews, and does its API provide the required invoice data under practical permissions and plan constraints?
6. Should PDF/document parsing be in the first MVP, or is CSV sufficient to demonstrate the workflow reliably?
7. What constitutes an unambiguous '15th' payment date, and what happens when the customer uses a date without a month or timezone?
8. What is the default automation mode at initial signup: manual review or opt-in bounded automation? Recommended default: manual review, with explicit opt-in for safe snoozes.

9. What data can be retained, for how long, and under which provider/processor terms? Obtain a documented data-flow and retention decision.
10. What pricing model and segment will be tested in paid pilots? Do not finalize pricing from assumptions alone.

Critical dependency: the hardest feasibility work is obtaining narrow, reliable access to invoice data, payment evidence, and customer email replies while maintaining trust and preventing false reminders. The LLM classifier is only one component of that system.

13 / DOWNSTREAM DOCUMENTS

Seven documents to create from this foundation

Use this concept brief as the shared baseline. Each downstream document should inherit the decisions here and turn them into a more specific artifact. If new research contradicts the concept, update the source of truth rather than preserving an assumption for consistency.

DOCUMENT	WHAT IT MUST CONTAIN	SOURCE SECTIONS
1. Product Proposal Document (PPD)	Purpose, vision, product thesis, problem context, target segment, value proposition, strategic boundaries, differentiators, assumptions, risks, and why the team should build it.	Use Sections 1-5, 8, and 12.
2. Market and Competitive Research Document	Customer discovery findings, competitor feature matrix, alternative workflows, segment sizing method, interview evidence, willingness to pay, and validation gaps.	Use Sections 2, 5, 11, and 12; replace hypotheses with sourced research.
3. Product Requirements Document (PRD)	Personas, jobs to be done, user stories, functional and non-functional requirements, workflows, scope, priorities, permissions, success metrics, and acceptance criteria.	Use Sections 2-4, 8-11.
4. UI/UX Design Document	Information architecture, exception-first dashboard, invoice detail, evidence view, mailbox connection, approval flows, policy configuration, empty/error/degraded states, and accessibility.	Use Sections 3-4, 8-10.
5. System Architecture Document	System context, components, trust boundaries, integration topology, tenant isolation, data flows, deployment model, resilience, observability, and security architecture.	Use Sections 6-8 and 12.
6. Technical Specification	Database schema, API contracts, webhook event handling, OAuth scopes, reconciliation algorithm, AI JSON schemas, scheduler/idempotency, state transitions, tests, and provider-specific implementation details.	Use Sections 6-7, 9, 10, and 11; verify every external API assumption.
7. Business Model and Go-to-Market Plan	ICP, buyer, pricing hypotheses, channels, partnership strategy, onboarding, pilot design, CAC/LTV assumptions, retention drivers, operating costs, and launch milestones.	Use Sections 2, 5, 10-12; do not treat suggested prices or segment ranges as validated.

Recommended document governance

- Maintain one shared glossary for invoice state, payment evidence, reconciliation match, commitment, dispute, snooze, approval, and automated action.
- Use a decision log for any change to product boundaries, default automation, payment matching, or mailbox permissions.
- Label every statement as validated fact, sourced external requirement, design decision, assumption, or open question.
- Keep a traceability chain from problem -> user story -> requirement -> design -> API/data model -> test case -> success metric.
- Do not promote an MVP limitation into a permanent product promise. Conversely, do not promise a capability that is only a future roadmap item.

14 / REFERENCES

Official sources and verification notes

The following sources were reviewed on 03 November 2026 for current integration and policy cautions. Provider documentation and laws can change. Re-check the exact API version, scopes, app-review requirements, regional availability, contracts, and applicability during implementation. This reference list supports technical planning; it is not legal advice.

[1] Google for Developers - Gmail API: users.messages.send

https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages/send

Documents the Gmail API send method and accepted authorization scopes.

[2] Google for Developers - Choose Gmail API scopes

https://developers.google.com/workspace/gmail/api/auth/scopes

Explains Gmail OAuth scope categories, least-privilege guidance, sensitive/restricted scopes, and verification considerations.

[3] Google for Developers - Configure push notifications with the Gmail API

https://developers.google.com/workspace/gmail/api/guides/push

Explains Gmail mailbox watch and Pub/Sub notifications, watch renewal, and reliability considerations including delayed or dropped notifications.

[4] Microsoft Learn - Send mail using Microsoft Graph

https://learn.microsoft.com/en-us/graph/api/user-sendmail?view=graph-rest-1.0

Documents the Graph sendMail endpoint and Mail.Send permission; accepted submission is not a guarantee of completed delivery.

[5] Microsoft Learn - Get subscription / Outlook message change notifications

https://learn.microsoft.com/en-us/graph/api/subscription-get?view=graph-rest-1.0

Explains that message change-notification subscriptions require read scope for the subscribed message resource.

[6] PayPal Developer - Integrate webhooks

https://developer.paypal.com/api/rest/webhooks/rest/

Explains webhook subscription, message verification, and listener implementation requirements.

[7] PayPal Developer - Webhooks overview

https://developer.paypal.com/api/rest/webhooks/

Explains event subscriptions, delivery, verification, and that events are associated with the relevant REST application.

[8] Press Information Bureau / Government of India - DPDP Rules, 2025 notified

https://www.pib.gov.in/PressNoteDetails.aspx?ModuleId=3&NoteId=156054&lang=2&reg=48

Government announcement on notification of the DPDP Rules on 14 November 2025 and the principles of the Indian data-protection framework.

Source-to-design cautions

- Google API scopes vary in sensitivity and restrictions. The exact mix required for send-only versus reply-reading must be designed deliberately and reviewed against Google's current user-data policy.
- Gmail push notifications require Pub/Sub setup and periodic watch renewal. A robust system needs history synchronization and fallback polling or reconciliation because notification delivery is not a complete correctness guarantee.
- Microsoft Graph message subscriptions require read access to message resources. Mail.Send alone should not be represented as sufficient to parse incoming replies.

- PayPal webhooks must be verified. The system should test which user-account transactions can be accessed through the specific approved PayPal integration and should not treat webhook absence as a complete statement of non-payment.
- The DPDP Act and Rules need a product-specific legal applicability review. Determine roles, notices, consent/lawful basis, processor contracts, retention, rights handling, security safeguards, cross-border processing, and applicable commencement timelines with counsel.

FINAL RECOMMENDATION

Final product recommendation: build CollectIQ as a privacy-conscious, relationship-aware receivables operations copilot. Win on payment truth, conversation-to-workflow intelligence, and safe exception resolution - not on the number of reminders sent.

End of concept foundation. This document is designed to seed the seven downstream product documents; it should be updated as customer interviews, competitor tests, API proofs, and security reviews replace assumptions with evidence.